

DOCUMENTO

Vector

vector.md

GERADO EM
23/08/2026 às 16:11

Sumário

Vector	3
1. Por que existe o vector?	3
2. Criando um vector	3
3. As operações do dia a dia	4
Adicionar no fim: <code>push_back</code>	4
Tamanho: <code>size</code>	4
Acessar um elemento: <code>[]</code>	4
Remover do fim: <code>pop_back</code>	4
Esvaziar: <code>clear</code>	4
4. Lendo e imprimindo um vector	5
5. Tabela de funções do vector	6
Tamanho e estado	6
Acesso a elementos	6
Inserir e remover	6
Posições (iteradores)	7
Capacidade (raramente necessário)	7
6. Funções de <code><algorithm></code> que valem ouro	7
7. Vector de duas dimensões (matriz)	9
8. Vector em funções	9
9. Erros comuns	10
10. Exemplo completo	11
Resumo rápido	11

Vector

Este material continua a Introdução ao C++. Aqui começamos a usar a **STL** (*Standard Template Library*), a biblioteca pronta que vem com o C++. A primeira ferramenta dela — e a mais usada em competições — é o `vector`.

1. Por que existe o `vector`?

Com arrays comuns, você precisa decidir o tamanho **antes** de rodar o programa:

```
int v[100];    // e se o problema pedir 101 números? E se pedir só 3?
```

cpp

O `vector` resolve isso. Ele é uma lista de elementos do mesmo tipo que:

- **cresce e diminui** durante a execução;
- **sabe o próprio tamanho** (`v.size()`);
- pode ser **copiado, comparado e devolvido por uma função**, coisas que um array comum não faz;
- é usado exatamente como um array: `v[0]`, `v[1]`, `v[2]`...

Pense nele como uma “fila de caixas numeradas” na qual você pode encaixar uma caixa nova no fim sempre que precisar.

NOTA

O `vector` faz parte do cabeçalho `<vector>`, mas como usamos `#include <bits/stdc++.h>`, ele já vem junto — não precisa incluir nada a mais.

2. Criando um vector

```
vector<int> a;                // vazio, tamanho 0
vector<int> b(10);            // 10 inteiros, todos valendo 0
vector<int> c(10, 7);         // 10 inteiros, todos valendo 7
vector<int> d = {4, 8, 15, 16}; // já com esses valores
vector<int> e = d;            // cópia independente de d

vector<double> notas;         // funciona com qualquer tipo
vector<string> nomes;
vector<char> letras;
```

cpp

O que vai entre `< >` é o **tipo** dos elementos. `vector<int>` guarda inteiros, `vector<string>` guarda strings, e assim por diante.

ATENÇÃO

`vector<int> b(10);` cria 10 elementos. `vector<int> b = {10};` cria **um** elemento valendo 10. Parecem iguais, mas não são.

3. As operações do dia a dia

Adicionar no fim: `push_back`

```
vector<int> v;      // v = {}  
v.push_back(5);    // v = {5}  
v.push_back(3);    // v = {5, 3}  
v.push_back(9);    // v = {5, 3, 9}
```

cpp

Tamanho: `size`

```
cout << v.size() << "\n";    // 3
```

cpp

Acessar um elemento: `[ ]`

```
cout << v[0] << "\n";        // 5  (o primeiro é o índice 0!)  
cout << v[2] << "\n";        // 9  (o último é o índice size() - 1)  
v[1] = 100;                   // v = {5, 100, 9}
```

cpp

Remover do fim: `pop_back`

```
v.pop_back();                // v = {5, 100}
```

cpp

Esvaziar: `clear`

```
v.clear();                   // v = {}, size() vira 0
```

4. Lendo e imprimindo um vector

Quando o problema diz quantos números virão:

```
int n;
cin >> n;

vector<int> v(n);           // já cria com n posições
for (int i = 0; i < n; i++) cin >> v[i];
```

cpp

Quando você **não** sabe quantos virão:

```
vector<int> v;
int x;
while (cin >> x) v.push_back(x);    // lê até a entrada acabar
```

cpp

Imprimindo:

```
for (int i = 0; i < (int) v.size(); i++) {
    cout << v[i];
    if (i + 1 < (int) v.size()) cout << " ";    // espaço só entre os números
}
cout << "\n";
```

cpp

Existe uma forma mais curta de percorrer, o **for baseado em intervalo**:

```
for (int x : v) cout << x << " ";    // lê cada elemento como cópia
cout << "\n";

for (int &x : v) x *= 2;               // com & você altera o vector
```

cpp

ATENÇÃO

`v.size()` devolve um tipo **sem sinal**. Se você escrever `for (int i = 0; i <= v.size() - 1; i++)` com o vector vazio, `v.size() - 1` não vira `-1`: vira um número gigante e o laço roda para sempre. Por isso usamos `(int) v.size()` ou a forma `i < v.size()`.

5. Tabela de funções do `vector`

Considere `vector<int> v;` em todos os exemplos.

Tamanho e estado

Função	O que faz	Exemplo
<code>v.size()</code>	quantos elementos há	<code>if (v.size() == 0) ...</code>
<code>v.empty()</code>	true se está vazio	<code>if (v.empty()) ...</code>
<code>v.resize(k)</code>	passa a ter exatamente <code>k</code> elementos (sobra vira 0, excesso é cortado)	<code>v.resize(5);</code>
<code>v.resize(k, x)</code>	idem, preenchendo as novas posições com <code>x</code>	<code>v.resize(5, -1);</code>
<code>v.clear()</code>	remove tudo	<code>v.clear();</code>
<code>v.assign(k, x)</code>	substitui todo o conteúdo por <code>k</code> cópias de <code>x</code>	<code>v.assign(n, 0);</code>

Acesso a elementos

Função	O que faz	Exemplo
<code>v[i]</code>	elemento da posição <code>i</code> (não confere se existe)	<code>v[3]</code>
<code>v.at(i)</code>	igual, mas acusa erro se <code>i</code> for inválido	<code>v.at(3)</code>
<code>v.front()</code>	primeiro elemento (mesmo que <code>v[0]</code>)	<code>cout << v.front();</code>
<code>v.back()</code>	último elemento (mesmo que <code>v[v.size()-1]</code>)	<code>cout << v.back();</code>

Inserir e remover

Função	O que faz	Exemplo
<code>v.push_back(x)</code>	acrescenta <code>x</code> no fim	<code>v.push_back(7);</code>
<code>v.pop_back()</code>	remove o último	<code>v.pop_back();</code>
<code>v.emplace_back(x)</code>	como <code>push_back</code> , porém constrói o elemento no lugar (mais rápido para tipos grandes)	<code>v.emplace_back(7);</code>
<code>v.insert(v.begin() + i, x)</code>	insere <code>x</code> na posição <code>i</code> , empurrando o resto	<code>v.insert(v.begin()+2, 9);</code>

Função	O que faz	Exemplo
<code>v.erase(v.begin() + i)</code>	apaga o elemento da posição <code>i</code>	<code>v.erase(v.begin()+2);</code>
<code>v.erase(v.begin()+i, v.begin()+j)</code>	apaga do índice <code>i</code> até <code>j-1</code>	<code>v.erase(v.begin(), v.begin()+3);</code>
<code>v.swap(w)</code>	troca o conteúdo de dois vectors	<code>v.swap(w);</code>

ATENÇÃO

`insert` e `erase` no **meio** do vector precisam deslocar todos os elementos seguintes. Fazer isso dentro de um laço grande deixa o programa lento ($O(n)$ por operação). Sempre que possível, adicione e remova pelo fim, com `push_back` / `pop_back`, que são instantâneos.

Posições (iteradores)

Um **iterador** é uma forma de apontar para uma posição do vector. Você o usará principalmente para passar intervalos às funções da próxima seção.

Função	O que faz
<code>v.begin()</code>	aponta para o primeiro elemento
<code>v.end()</code>	aponta para depois do último (não é um elemento válido)
<code>v.rbegin()</code> / <code>v.rend()</code>	os mesmos, na ordem invertida

A dupla `v.begin()`, `v.end()` significa, na prática, “o vector inteiro”.

Capacidade (raramente necessário)

Função	O que faz
<code>v.capacity()</code>	quanto espaço já está reservado na memória
<code>v.reserve(k)</code>	reserva espaço para <code>k</code> elementos de uma vez, evitando realocações
<code>v.shrink_to_fit()</code>	devolve a memória sobrando

Isso só importa quando você vai inserir milhões de elementos; para a maioria dos problemas, ignore.

6. Funções de `<algorithm>` que valem ouro

Não são funções do `vector`, mas trabalham com ele e resolvem meio problema sozinhas. Todas recebem o intervalo `v.begin()`, `v.end()`.

Função	O que faz
<code>sort(v.begin(), v.end())</code>	ordena em ordem crescente
<code>sort(v.rbegin(), v.rend())</code>	ordena em ordem decrescente
<code>reverse(v.begin(), v.end())</code>	inverte a ordem dos elementos
<code>max_element(v.begin(), v.end())</code>	aponta para o maior elemento
<code>min_element(v.begin(), v.end())</code>	aponta para o menor elemento
<code>count(v.begin(), v.end(), x)</code>	quantas vezes x aparece
<code>find(v.begin(), v.end(), x)</code>	procura x; devolve <code>v.end()</code> se não achar
<code>accumulate(v.begin(), v.end(), 0LL)</code>	soma todos os elementos
<code>binary_search(v.begin(), v.end(), x)</code>	diz se x existe (exige vector ordenado)
<code>lower_bound(v.begin(), v.end(), x)</code>	primeira posição $\geq x$ (exige ordenado)
<code>unique(v.begin(), v.end())</code>	junta repetidos vizinhos no início (use após <code>sort</code>)

Exemplos:

```
vector<int> v = {5, 2, 9, 2, 7};

sort(v.begin(), v.end());           // v = {2, 2, 5, 7, 9}

cout << *max_element(v.begin(), v.end()); // 9   (o * pega o valor apontado)
cout << count(v.begin(), v.end(), 2);     // 2

long long soma = accumulate(v.begin(), v.end(), 0LL); // 25

reverse(v.begin(), v.end());         // v = {9, 7, 5, 2, 2}
```

cpp

Removendo elementos repetidos (padrão muito comum):

```
sort(v.begin(), v.end());
v.erase(unique(v.begin(), v.end()), v.end()); // agora v só tem valores distintos
```

cpp

Verificando se um valor está presente:

```
if (find(v.begin(), v.end(), 7) != v.end()) cout << "achei\n";
```

NOTA

Use `0LL` (e não `0`) em `accumulate` quando a soma puder passar do limite do `int` — esse terceiro argumento define o tipo da soma.

7. Vector de duas dimensões (matriz)

Um `vector` pode guardar outros `vectors`. É assim que se faz uma matriz de tamanho definido durante a execução:

```
int n, m;
cin >> n >> m;

vector<vector<int>> mat(n, vector<int>(m, 0)); // n linhas, m colunas, tudo 0

for (int i = 0; i < n; i++)
    for (int j = 0; j < m; j++)
        cin >> mat[i][j];
```

cpp

Lendo em voz alta: “um vector com `n` posições, e cada posição é um vector de `m` inteiros valendo 0”.

- `mat.size()` → número de linhas
- `mat[0].size()` → número de colunas

8. Vector em funções

Diferente de arrays, um `vector` pode ser passado e devolvido normalmente:

```
// recebe uma CÓPIA – alterações não afetam o original
int soma (vector<int> v) {
    int s = 0;
    for (int x : v) s += x;
    return s;
}

// recebe por REFERÊNCIA – sem cópia e podendo alterar o original
void dobra (vector<int> &v) {
    for (int &x : v) x *= 2;
}

// referência constante: sem cópia, mas proibido alterar (o ideal para só ler)
int maior (const vector<int> &v) {
    return *max_element(v.begin(), v.end());
}

// devolvendo um vector
vector<int> pares (const vector<int> &v) {
    vector<int> r;
    for (int x : v) if (x % 2 == 0) r.push_back(x);
    return r;
}
```

cpp

ATENÇÃO

Copiar um vector grande custa tempo. Em competição, passe sempre por referência (&) — use `const vector<int> &v` quando a função só precisa ler.

9. Erros comuns

Erro	O que acontece
<code>v[5]</code> com <code>v.size() == 3</code>	lixo ou <i>runtime error</i> ; o vector não cresce sozinho no acesso
Usar <code>v[i] = x</code> para adicionar	não funciona se a posição não existir; use <code>push_back</code>
<code>v.size() - 1</code> com vector vazio	vira um número enorme (tipo sem sinal)
<code>binary_search</code> sem ordenar antes	resposta errada, sem aviso
Guardar <code>v.back()</code> e depois dar <code>push_back</code>	referências antigas podem ficar inválidas após o vector crescer
Passar vector grande por valor	<i>Time Limit Exceeded</i> por causa das cópias

10. Exemplo completo

Problema: ler `n` notas, imprimir a média, a maior nota e as notas em ordem crescente.

```
#include <bits/stdc++.h>
using namespace std;

int main () {
    ios_base::sync_with_stdio(false);
    cin.tie(NULL);

    int n;
    cin >> n;

    vector<double> notas(n);
    for (int i = 0; i < n; i++) cin >> notas[i];

    double soma = accumulate(notas.begin(), notas.end(), 0.0);
    double media = soma / n;
    double maior = *max_element(notas.begin(), notas.end());

    sort(notas.begin(), notas.end());

    cout << fixed << setprecision(2);
    cout << "Media: " << media << "\n";
    cout << "Maior: " << maior << "\n";
    cout << "Ordenadas:";
    for (double x : notas) cout << " " << x;
    cout << "\n";

    return 0;
}
```

cpp

Resumo rápido

```
vector<int> v;           // cria
v.push_back(x);         // adiciona no fim
v.pop_back();           // remove do fim
v[i]                    // acessa
v.size()                 // tamanho
v.empty()                // está vazio?
v.clear()                // esvazia
v.front() / v.back()     // primeiro / último
sort(v.begin(), v.end()); // ordena
```

cpp